

Iterator

An iterator object is one that implements the `next()` method. The `next()` method returns an object {value: Any, done: Boolean}. It fetches values from the collection, one at a time. When it reaches the end, the value of the `done` property is `true`. When this is the case, the `value` property is undefined. The method `next()` is a zero arguments function. Let us look at an example, which is shown below:

```
1. let iterable = ['a', 'b', 'c'];
2.
3. let iterator = iterable[Symbol.iterator]();
4. iterator.next(); // {value: 'a', done: false}
5. iterator.next(); // {value: 'b', done: false}
6. iterator.next(); // {value: 'c', done: false}
7. iterator.next(); // {value: undefined, done: true}
```

Example 2: An iterable defined and traversed

After understanding the definitions of iterable and iterator, let us look at a program that makes the concept clear. The following program generates Fibonacci numbers under 50. *Fib* is an iterable object and also an iterator. It implements [*Symbol.iterator*] and the `next()` function.

```
1. let fib = {
2.   [Symbol.iterator]: function() {
3.     let pre = 0, cur = 1, value;
4.     return {
5.       next: function() {
6.         value = pre;
7.         pre = cur;
8.         cur = pre + cur;
9.         return {done: false, value: value};
10.      }
11.    }
12.  }
13. }
14.
15. for (var n of fib) {
16.   // truncate the sequence at 50
17.   if (n > 50)
18.     break;
19.   console.log(n);
20. }
```

Example 3: Fibonacci iterable object

The above example can be rewritten using other ES6 features, as shown below:

```
1. let fib = {
2.   [Symbol.iterator]() {
3.     let pre = 0, cur = 1, value;
4.     return {
5.       next() {
```

```
6.       [value, pre, cur] = [pre, cur, pre + cur];
7.       return {value};
8.     }
9.   }
10. }
11. }
12.
13. for (let n of fib) {
14.   // truncate the sequence at 50
15.   if (n > 50)
16.     break;
17.   console.log(n);
18. }
```

Example 4: Fibonacci iterable object (the ES6 way)

Note the removal of the `function` keyword in Lines 2 and 5.

In Line 6, we used the destructuring feature to assign multiple values at a time.

In Line 7, we used the shorthand property notation. Since the property and the local variable name are the same, we just use 'value'.

The above example is available online at <http://www.es6fiddle.net/in4vk7ng/>. Take a look at it.

Another point to note in the above example is that the Fibonacci object generates an infinite sequence. The control to define the end of sequence generation is outside the function. In cases of built-in iterators like arrays, the array length defines the end of the collection.

In JavaScript, we can list Symbols associated with an object as shown below:

```
> Object.getOwnPropertySymbols(fib)
```

ES6 support in browser environments

What's amazing is that in the last eight months, the support for ES6 features has steadily increased. The top three browsers, Chrome, Firefox and Microsoft Edge, now have 90 per cent or more support. Google Chrome is leading with 97 per cent in version 52.

Next month, we will learn about an interesting new feature called generators, also called co-routines. Generators break the flow of control and resume from the same place. Internally, the language also remembers the context of execution. This feature, when used in conjunction with iterators, is very powerful. 

References

- [1] ES6 Iterators, Generators, and Iterables: <https://blog.domenic.me/es6-iterators-generators-and-iterables/>
- [2] JavaScript looping evolution: <http://developer.telerik.com/featured/the-javascript-looping-evolution/>
- [3] Kangax support matrix: <http://kangax.github.io/compat-table/es6/>

By: Janardan Revuru

The author works as an engineering manager at Hewlett Packard Enterprise, Bengaluru. He is a co-organiser of the 'JavaScript meetup - Bangalore' group at meetup.com. <http://www.meetup.com/JavaScript-Meetup-Bangalore/>.